

Forum Application

Documentație Tehnică — Partea 1

Studenti: Tudor Urzica, Popa Denis, Vancea Adelin

Coordonator: Bogdan Bindea

Universitatea Tehnică din Cluj-Napoca
Facultatea de Automatică și Calculatoare

Cluj-Napoca, 2026

Cuprins

1	Introducere	3
1.1	Scopul proiectului	3
1.2	Obiective principale	3
1.3	Fluxul principal al aplicației	3
2	Stiva de Tehnologii	4
2.1	Spring Boot 3.2.4	4
2.2	Lombok	4
2.3	Hibernate / JPA	5
2.4	Angular 21	5
2.5	SendGrid (Notificări E-mail)	5
2.6	Twilio (Notificări SMS)	6
2.7	Teste Postman	6
2.8	Mockito (Teste Unitare Backend)	7
2.9	Cypress (Teste E2E și de Integrare)	7
2.9.1	Teste API (Unit)	7
2.9.2	Teste de Componente UI	7
2.9.3	Teste de Integrare	8
3	Arhitectura Aplicației	8
3.1	Arhitectura generală	8
3.2	Structura Backend (Spring Boot)	8
3.2.1	Pachetele Java	9
3.3	Structura Frontend (Angular)	9
3.4	Securitate	9

3.5	Comunicarea inter-servicii	10
4	Principalele Entități	10
4.1	Diagrama Entitate-Relație	10
4.2	Entitatea User	10
4.3	Entitatea Question	11
4.4	Entitatea Answer	12
4.5	Entitatea Tag și relația Many-to-Many	12
4.6	Entitățile de Vot: QuestionVote și AnswerVote	13
4.7	Ierarhia Attachment	13
4.8	Enum-urile Role și Status	14
4.9	Sistemul de Scor (Reputație)	14
5	Servicii REST și API Endpoints (Backend)	14
5.1	Data Transfer Objects (DTOs)	14
5.2	Endpoint-uri REST (Controllere)	15
5.2.1	QuestionController	15
5.2.2	AnswerController	15
5.2.3	UserController	15
5.2.4	AuthController	15
6	Aplicația Frontend (Angular)	16
6.1	Structura și Rutare	16
6.2	Componente Principale	16
6.2.1	QuestionList	16
6.2.2	AnswersList	16
6.2.3	AuthInterceptor	17
7	Servicii de Notificare: SendGrid și Twilio	17
8	Integrare Microservicii (Backend-Second)	17
9	Concluzii	17

1 Introducere

1.1 Scopul proiectului

Forum Application este o platformă web de tip *Questions & Answers* (Q&A), similară conceptual cu Stack Overflow, dezvoltată ca proiect în cadrul disciplinei *Proiectare Software*. Aplicația permite utilizatorilor să posteze întrebări, să primească răspunsuri de la comunitate, să voteze conținutul și să interacționeze într-un mediu colaborativ.

1.2 Obiective principale

- **Autentificare și autorizare** — sistem de înregistrare și login cu roluri (USER, ADMIN), implementat cu Spring Security și HTTP Basic Authentication.
- **Gestionarea întrebărilor** — utilizatorii pot crea, edita, șterge și filtra întrebări, cu suport pentru tag-uri și atașamente de imagini.
- **Sistem de răspunsuri** — răspunsurile sunt asociate întrebărilor, cu posibilitatea de acceptare a unui răspuns ca soluție definitivă.
- **Sistem de votare** — mecanism de *like/dislike* atât pentru întrebări cât și pentru răspunsuri, cu logică de tip *toggle* (un utilizator poate schimba votul).
- **Sistem de reputație** — scor calculat pe baza voturilor primite și acordate, cu ponderi diferite per tip de acțiune.
- **Notificări externe** — integrare cu **SendGrid** (e-mail) și **Twilio** (SMS) pentru notificarea utilizatorilor blocați.
- **Arhitectură microservicii** — gestionarea tag-urilor se face printr-un microserviciu separat (**backend-second**, port 8081).
- **Testare comprehensivă** — teste unitare (JUnit + Mockito), teste de integrare API (Cypress), teste end-to-end UI (Cypress), teste de componente Angular.

1.3 Fluxul principal al aplicației

1. Utilizatorul se înregistrează (`/auth/register`) și primește rolul implicit **USER**.
2. După autentificare (`/auth/login`), utilizatorul este redirecționat către lista de întrebări.
3. Utilizatorul poate posta o întrebare (cu titlu, text, tag-uri și imagini opționale). Statusul inițial este **RECEIVED**.
4. Alți utilizatori pot răspunde la întrebare. La primul răspuns, statusul devine **IN_PROGRESS**.
5. Utilizatorii votează (*like/dislike*) întrebări și răspunsuri; voturile influențează scorul de reputație.
6. Autorul întrebării poate accepta un răspuns, trecând statusul la **RESOLVED**.
7. Administratorii pot bloca/debloca utilizatori; la blocare se trimit notificări prin e-mail (SendGrid) și SMS (Twilio).

2 Stiva de Tehnologii

2.1 Spring Boot 3.2.4

Spring Boot reprezintă fundamentul backend-ului aplicației, oferind:

- **Auto-configurare** — detectează și configurează automat dependențele din class-path.
- **Server web încorporat** — Tomcat integrat, eliminând necesitatea unui deployment extern.
- **Spring MVC** — framework pentru construcția API-urilor RESTful prin adnotări `@RestController` și `@RequestMapping`.
- **Spring Data JPA** — simplificarea interacțiunii cu baza de date prin interfețe repository.
- **Spring Security** — securizarea endpoint-urilor cu HTTP Basic Authentication.
- **Spring Validation** — validarea datelor de intrare prin adnotări Jakarta Bean Validation.
- **Spring Mail** — suport pentru trimiterea de e-mailuri.
- **@EnableAsync** — suport pentru execuția asincronă a notificărilor.

Versiunea de Java utilizată este **Java 21**.

2.2 Lombok

Project Lombok reduce semnificativ boilerplate-ul din codul Java prin adnotări procesate la compilare: Lombok este configurat ca *annotation processor* în `maven-compiler-plugin`

Adnotare	Descriere
<code>@Data</code>	Generează <code>getters</code> , <code>setters</code> , <code>toString()</code> , <code>equals()</code> , <code>hashCode()</code>
<code>@Builder</code>	Implementează design pattern-ul Builder pentru construcția obiectelor
<code>@NoArgsConstructor</code>	Generează constructor fără argumente
<code>@AllArgsConstructor</code>	Generează constructor cu toate câmpurile ca argumente
<code>@SuperBuilder</code>	Extinde <code>@Builder</code> pentru ierarhii de moștenire (utilizat în <code>Attachment</code>)
<code>@EqualsAndHashCode</code>	Controlează generarea <code>equals/hashCode</code> cu opțiunea <code>callSuper</code>
<code>@Builder.Default</code>	Permite setarea unei valori implicite în pattern-ul Builder

Tabela 1: Adnotările Lombok utilizate în proiect

și este exclus din bundle-ul final de producție prin `spring-boot-maven-plugin`.

2.3 Hibernate / JPA

Hibernate, în calitate de implementare JPA, gestionează maparea obiectuală-relațională:

- **Entități JPA** — clasele din pachetul `entity` sunt adnotate cu `@Entity` și mapate către tabele MySQL.
- **Relații** — `@OneToMany`, `@ManyToOne`, `@ManyToMany` cu tabele de joncțiune (`question_tags`).
- **Fetch Strategies** — `FetchType.LAZY` pentru optimizarea încărcării.
- **Cascade Operations** — `CascadeType.ALL` pentru operațiile în cascadă, `orphanRemoval=true` pentru ștergerea automată a entităților orfane.
- **@CreationTimestamp** — generare automată a timestamp-ului la crearea entității.
- **@MappedSuperclass** — moștenire la nivel de entitate (`Attachment` → `QuestionImage`, `AnswerImage`).
- **DDL Auto-Update** — `spring.jpa.hibernate.ddl-auto=update` permite crearea/actualizarea automată a schemei.
- **Baza de date** — MySQL, URL: `jdbc:mysql://localhost:3306/forum-app`.

2.4 Angular 21

Frontend-ul este construit cu **Angular 21**, utilizând o arhitectură bazată pe:

- **Standalone Components** — toate componentele sunt *standalone*, fără module Angular tradiționale.
- **Signals** — sistemul reactiv Angular Signals pentru gestionarea stării (înlocuiește parțial RxJS Observables).
- **Computed Signals** — semnale derivate pentru filtrare și sortare automată.
- **HttpClient cu Interceptors** — `authInterceptor` adaugă automat headerul `Authorization: Basic` pe fiecare request autentificat.
- **Router Guards** — `authGuard` (protejează rutele autentificate) și `guestGuard` (redirecționează utilizatorii logați).
- **Routing** — 7 rute definite cu redirecționare implicită către pagina de login.
- **SCSS** — pre-procesor CSS pentru stilizare.
- **SessionStorage** — stocarea credențialelor și a rolului utilizatorului curent.

2.5 SendGrid (Notificări E-mail)

SendGrid (v4.9.3) este integrat pentru trimiterea de e-mailuri automate atunci când un cont de utilizator este blocat. Implementarea se află în clasa `NotificationService`:

```
1 Email sender = new Email(sendGridFromEmail);
2 Email receiver = new Email(user.getEmail());
3 Content content = new Content("text/plain",
4     "Your account has been blocked. Please contact support.");
5 Mail mail = new Mail(sender, "Account blocked", receiver, content);
```

```
6 SendGrid sendGrid = new SendGrid(sendGridApiKey);
7 Request request = new Request();
8 request.setMethod(Method.POST);
9 request.setEndpoint("mail/send");
10 request.setBody(mail.build());
11 Response response = sendGrid.api(request);
```

Listing 1: Trimiterea e-mailului prin SendGrid

Configurarea se face prin variabile de mediu:

- `SENDGRID_ENABLED` — activare/dezactivare (implicit `false`)
- `SENDGRID_API_KEY` — cheia API SendGrid
- `SENDGRID_FROM_EMAIL` — adresa expeditorului

2.6 Twilio (Notificări SMS)

Twilio (v9.14.0) este utilizat pentru trimiterea de SMS-uri când un utilizator blocat încearcă autentificarea:

```
1 Twilio.init(twilioAccountSid, twilioAuthToken);
2 Message message = Message.creator(
3     new PhoneNumber(user.getPhone()),
4     new PhoneNumber(twilioFromNumber),
5     "Your account has been blocked. Please contact support."
6 ).create();
```

Listing 2: Trimiterea SMS-ului prin Twilio

Ambele servicii (SendGrid și Twilio) sunt apelate **asincron** (adnotarea `@Async`) din `CustomUserDetailsService` la fiecare tentativă de login a unui utilizator blocat, evitând blocarea thread-ului principal.

2.7 Teste Postman

Proiectul include **colecții Postman** organizate în directorul `backend/postman/collections/`, care acoperă testarea manuală a tuturor endpoint-urilor API REST:

- **CRUD Users** — înregistrare, citire, actualizare, ștergere utilizatori
- **CRUD Questions** — creare (cu/fără imagini), citire (după titlu, autor, ID), actualizare, ștergere, like/dislike
- **CRUD Answers** — creare, citire, actualizare, ștergere, like/dislike, acceptare
- **CRUD Tags** — creare, citire, actualizare, ștergere (prin microserviciul secundar)
- **Auth** — register, login
- **Admin Actions** — block/unblock users

Colecțiile Postman permit autentificarea HTTP Basic și testarea manuală rapidă a API-ului în timpul dezvoltării.

2.8 Mockito (Teste Unitare Backend)

Mockito este utilizat în testele unitare ale serviciilor backend:

```
1 @ExtendWith(MockitoExtension.class)
2 class QuestionServiceTest {
3     @Mock private QuestionRepository questionRepository;
4     @Mock private UserRepository userRepository;
5     @Mock private TagRepository tagRepository;
6     @InjectMocks private QuestionService questionService;
7     @Test
8     void testSaveQuestionSuccess() {
9         when(userRepository.findByUsername("user1"))
10            .thenReturn(testUser);
11         when(questionRepository.existsByTitle("Test"))
12            .thenReturn(false);
13         when(questionRepository.save(any()))
14            .thenReturn(testQuestion);
15         // ...
16     }
17 }
```

Listing 3: Exemplu de test Mockito — QuestionServiceTest

Testele unitare acoperă:

- **QuestionServiceTest** — 6 teste: salvare cu succes, titlu duplicat, găsim după titlu, ștergere, listare, titlu inexistent
- **AnswerServiceTest** — 5 teste: salvare, găsim după ID, ID inexistent, listare după întrebare, ștergere
- **UserServiceTest** — 4 teste: salvare, username duplicat, găsim după username, username inexistent
- **Controller Tests (WebMvcTest)** — 13 teste: QuestionController (5), AnswerController (4), UserController (4) — testează stratul HTTP cu mock-uri

2.9 Cypress (Teste E2E și de Integrare)

Cypress (v15.15) este utilizat pentru testarea end-to-end la nivel de API și UI:

2.9.1 Teste API (Unit)

- `question-controller.cy.js` — 8 teste: CRUD complet + votare
- `answer-controller.cy.js` — 9 teste: CRUD + votare + acceptare
- `user-controller.cy.js` — 8 teste: CRUD + blocare + scor
- `tag-controller.cy.js` — 5 teste: CRUD prin microserviciul pe portul 8081

2.9.2 Teste de Componente UI

- `navbar-login.cy.js` — 5 teste: randare formular login, navbar logat/delogat
- `question-list.cy.js` — 5 teste: afișare tabel, date, filtrare

- `user-profile.cy.js` — 4 teste: randare profil, contact, întrebări

2.9.3 Teste de Integrare

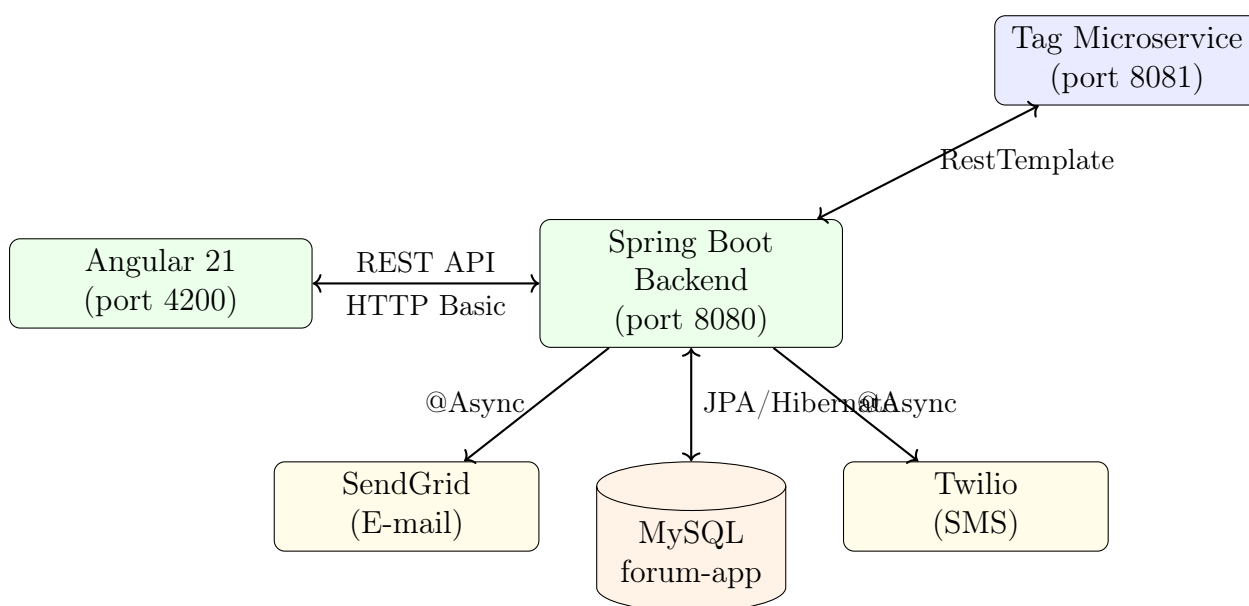
- `questions.cy.js`, `answers.cy.js`, `users.cy.js`, `tags.cy.js` — teste complete pe fluxuri API
- `frontend.cy.js` — smoke tests UI (login, redirecționare)
- `integration-flow.cy.js` — flux complet: creare întrebare → adăugare răspuns → votare → acceptare → verificare status **RESOLVED**

Total teste în proiect: ~90+ teste (JUnit + Mockito + Cypress).

3 Arhitectura Aplicației

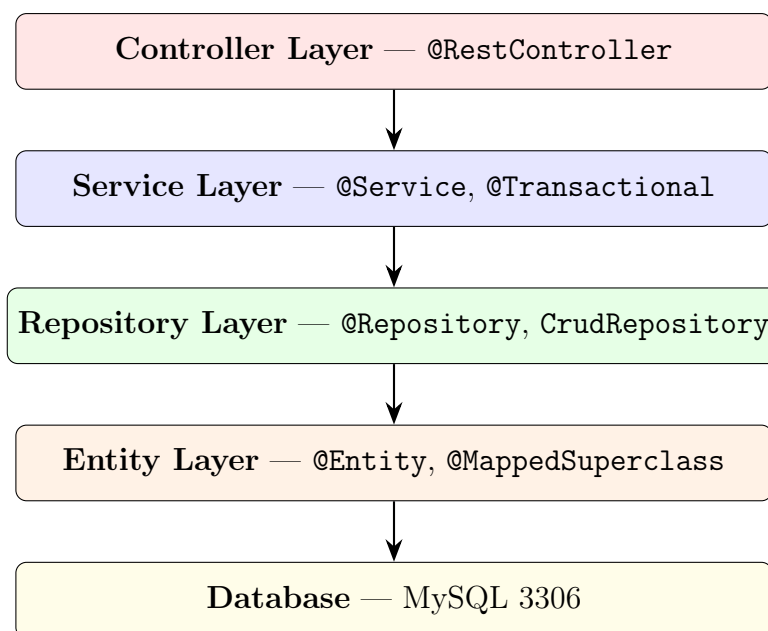
3.1 Arhitectura generală

Aplicația este construită pe o arhitectură **client-server** cu separare clară între frontend și backend, urmând modelul **Model-View-Controller (MVC)** pe partea de server și o arhitectură bazată pe **componente standalone** pe partea de client.



3.2 Structura Backend (Spring Boot)

Backend-ul urmează o arhitectură **stratificată** (layered architecture):



3.2.1 Pachetele Java

Pachet	Conținut
entity	Entitățile JPA: User, Question, Answer, Tag, QuestionVote, AnswerVote, QuestionImage, AnswerImage, Attachment, enum-urile Role, Status
controller	Controllerele REST: QuestionController, AnswerController, UserController, AuthController, TagController
service	Logica de business: QuestionService, AnswerService, UserService, TagService, NotificationService, TagMicroserviceClient, UserScoreService, CustomUserDetailsService
repository	Interfețe Spring Data: QuestionRepository, AnswerRepository, UserRepository, TagRepository, QuestionVoteRepository, AnswerVoteRepository
dto	Data Transfer Objects: QuestionDTO, AnswerDTO, UserDTO, TagDTO
config	Configurare: SecurityConfig, WebConfig, RestTemplateConfig

Tabela 2: Organizarea pachetelor Java

3.3 Structura Frontend (Angular)

3.4 Securitate

Configurarea securității (SecurityConfig.java) definește:

- **CORS** — permite origin `http://localhost:4200` cu metode GET, POST, PUT, DELETE, OPTIONS
- **CSRF dezactivat** — API-ul este stateless
- **Rute publice** — `/api/auth/register`, `/api/auth/login`, `/uploads/**`

Modul/Director	Componente
features/questions	QuestionList — pagină principală cu tabel de întrebări, filtre, creare/editare
features/answers	AnswersList — pagina de detaliu a unei întrebări cu lista de răspunsuri
features/auth	Login, Register — autentificare, guards, interceptor HTTP
features/users	UserList, UserProfile — administrare utilizatori, profil cu scor
shared/components	Navbar — bara de navigare cu link-uri condiționate de rol

Tabela 3: Organizarea frontend-ului Angular

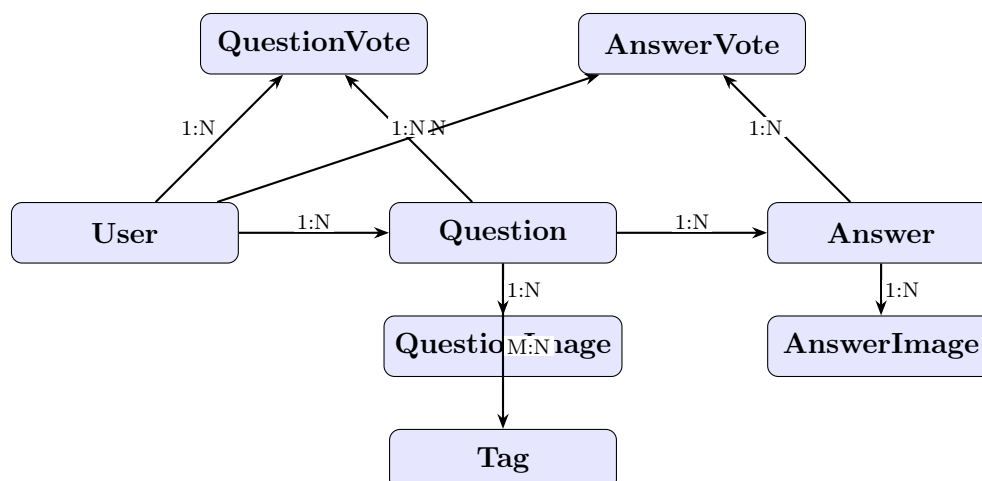
- **Rute admin** — `/api/users/*/block`, `/api/users/*/unblock` necesită rolul ADMIN
- **HTTP Basic** — cu entry point personalizat (403 pentru conturi blocate, 401 pentru neautentificare)
- **BCrypt** — parola este hash-uită cu `BCryptPasswordEncoder`

3.5 Comunicarea inter-servicii

`TagController` din backend-ul principal nu accesează direct baza de date pentru tag-uri, ci delegă operațiile către `TagMicroserviceClient`, care comunică prin **RestTemplate** cu microserviciul secundar (`backend-second`) la adresa `http://localhost:8081/api/tags`.

4 Principalele Entități

4.1 Diagrama Entitate-Relație



4.2 Entitatea User

```

1 @Entity
2 @Table(name="users")
3 @Data @AllArgsConstructor @NoArgsConstructor @Builder
4 public class User {

```

```

5  @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
6  @Column(name = "user_id")
7  private Long id;
8  @Column(name = "first_name") private String firstName;
9  @Column(name = "last_name") private String lastName;
10 @Column(name = "email", unique = true) private String email;
11 @Column(name = "phone") private String phone;
12 @Column(name = "username", unique = true, nullable = false)
13 private String username;
14 @Column(name = "password") private String password;
15 @Enumerated(EnumType.STRING)
16 @Column(name = "role") @Builder.Default
17 private Role role = Role.USER;
18 @Column(name = "access_restricted")
19 private boolean accessRestricted = false;
20 @OneToMany(mappedBy = "author", cascade = CascadeType.ALL,
21             orphanRemoval = true)
22 private List<Question> questions = new ArrayList<>();
23 }

```

Listing 4: Entitatea User

Câmpuri cheie:

- username — unic, obligatoriu, folosit ca identicator de autentificare
- password — stocat hash-uit cu BCrypt; în DTO marcat @JsonProperty(WRITE_ONLY)
- role — enum USER sau ADMIN
- accessRestricted — flag de blocare; influențează autentificarea și declanșează notificări

4.3 Entitatea Question

```

1  @Entity @Table(name="questions")
2  @Data @AllArgsConstructor @NoArgsConstructor @Builder
3  public class Question {
4      @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
5      @Column(name = "question_id") private Long id;
6      @ManyToOne(fetch = FetchType.LAZY)
7      @JoinColumn(name = "user_id", nullable = false)
8      private User author;
9      @Column(name = "title") private String title;
10 @Column(name = "text") private String text;
11 @Column(nullable = false, columnDefinition = "int default 0")
12 private int likes = 0;
13 @Column(nullable = false, columnDefinition = "int default 0")
14 private int dislikes = 0;
15 @CreationTimestamp
16 @Column(name = "date_and_time", updatable = false)
17 private LocalDateTime date;
18 @Enumerated(EnumType.STRING)
19 @Column(name = "status") private Status status;
20 @ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
21 @JoinTable(name = "question_tags",
22            joinColumns = @JoinColumn(name = "question_id"),
23            inverseJoinColumns = @JoinColumn(name = "tag_id"))

```

```

24 private List<Tag> tags = new ArrayList<>();
25 @OneToMany(mappedBy = "question", cascade = CascadeType.ALL,
26             orphanRemoval = true, fetch = FetchType.LAZY)
27 private List<Answer> answers = new ArrayList<>();
28 @OneToMany(mappedBy = "question", cascade = CascadeType.ALL,
29             orphanRemoval = true, fetch = FetchType.LAZY)
30 private List<QuestionImage> images = new ArrayList<>();
31 }

```

Listing 5: Entitatea Question

Status workflow:

1. RECEIVED — la creare
2. IN_PROGRESS — la primul răspuns
3. RESOLVED — când autorul acceptă un răspuns

4.4 Entitatea Answer

```

1 @Entity @Table(name="answers")
2 @Data @AllArgsConstructor @NoArgsConstructor @Builder
3 public class Answer {
4     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
5     @Column(name = "answer_id") private Long id;
6     @ManyToOne(fetch = FetchType.LAZY)
7     @JoinColumn(name = "user_id", nullable = false)
8     private User author;
9     @Column(name = "text") private String text;
10    private int likes = 0;
11    private int dislikes = 0;
12    private boolean accepted = false;
13    @CreationTimestamp
14    @Column(name = "date_and_time", updatable = false)
15    private LocalDateTime date;
16    @ManyToOne(fetch = FetchType.LAZY)
17    @JoinColumn(name = "question_id", nullable = false)
18    private Question question;
19    @OneToMany(mappedBy = "answer", cascade = CascadeType.ALL,
20              orphanRemoval = true)
21    private List<AnswerImage> images;
22 }

```

Listing 6: Entitatea Answer

Particularități:

- Câmpul `accepted` marchează răspunsul acceptat de autorul întrebării
- Doar autorul întrebării poate accepta un răspuns
- La acceptare, statusul întrebării devine `RESOLVED`

4.5 Entitatea Tag și relația Many-to-Many

```

1 @Entity @Table(name="tags")
2 @Data @AllArgsConstructor @NoArgsConstructor @Builder
3 public class Tag {
4     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
5     @Column(name = "tag_id") private Long id;
6     @Column(name = "label", unique = true, nullable = false)
7     private String label;
8     @ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
9     @JoinTable(name = "question_tags",
10         joinColumns = @JoinColumn(name = "tag_id"),
11         inverseJoinColumns = @JoinColumn(name = "question_id"))
12     private List<Question> questions = new ArrayList<>();
13 }

```

Listing 7: Entitatea Tag

Relația ManyToMany dintre Question și Tag utilizează tabela de joncțiune question_tags. Tag-urile sunt gestionate și printr-un **microserviciu separat** (backend-second, port 8081).

4.6 Entitățile de Vot: QuestionVote și AnswerVote

```

1 @Entity
2 @Table(name="question_votes", uniqueConstraints = {
3     @UniqueConstraint(columnNames = {"user_id", "question_id"})
4 })
5 public class QuestionVote {
6     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8     @ManyToOne(fetch = FetchType.LAZY)
9     @JoinColumn(name = "user_id", nullable = false)
10    private User user;
11    @ManyToOne(fetch = FetchType.LAZY)
12    @JoinColumn(name = "question_id", nullable = false)
13    private Question question;
14    @Column(nullable = false)
15    private boolean isLike;
16 }

```

Listing 8: Entitatea QuestionVote

Constrângerea de unicitate pe perechea (user_id, question_id) garantează că un utilizator poate acorda **un singur vot per întrebare/răspuns**. Logica de toggle din QuestionService / AnswerService permite schimbarea votului existent.

4.7 Ierarhia Attachment

Clasele QuestionImage și AnswerImage extind clasa abstractă Attachment (adnotată @MappedSuperclass):

```

1 @MappedSuperclass @Data @SuperBuilder
2 @NoArgsConstructor @AllArgsConstructor
3 public abstract class Attachment {
4     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
5     private Long id;

```

```

6   @Column(nullable = false)
7   private String imageUrl;
8 }

```

Listing 9: Clasa abstractă Attachment

Fișierele imagine sunt salvate pe disc în directoarele `uploads/questions/` și `uploads/answers/`, iar URL-urile relative sunt stocate în baza de date.

4.8 Enum-urile Role și Status

```

1 public enum Role { USER, ADMIN }
2 public enum Status { RECEIVED, IN_PROGRESS, RESOLVED }

```

Listing 10: Enum-urile Role și Status

4.9 Sistemul de Scor (Reputație)

Clasa `UserScoreService` calculează scorul de reputație pe baza formulei: Scorul este cal-

Acțiune	Punctaj
Upvote pe întrebare proprie	+2.5
Upvote pe răspuns propriu	+5.0
Downvote pe întrebare proprie	-1.5
Downvote pe răspuns propriu	-2.5
Acordarea unui downvote pe răspunsul altcuiva	-1.5

Tabela 4: Sistemul de punctaj pentru reputație

culat dinamic prin interogări JPQL în `QuestionVoteRepository` și `AnswerVoteRepository` și este inclus în fiecare `UserDTO` returnat de API.

5 Servicii REST și API Endpoints (Backend)

Arhitectura backend-ului folosește controlere REST pentru expunerea funcționalității. Pentru transferul de date, se utilizează clase **DTO** (Data Transfer Object) care încapsulează logica, protejând entitățile interne și limitând informațiile trimise către client (de exemplu, omiterea parolelor).

5.1 Data Transfer Objects (DTOs)

Proiectul implementează următoarele DTO-uri principale:

- **QuestionDTO** — Conține `questionId`, `title`, `text`, `likes`, `dislikes`, `date`, `status`, `authorName`, `authorScore`, lista de `tags`, lista de `imageUrls` și starea votului utilizatorului curent (`currentUserVote`).
- **AnswerDTO** — Conține identificatori pentru întrebare și răspuns, `authorName`, `text`, scoruri, `imageUrls`, starea votului și flag-ul `accepted`.

- **UserDTO** — Conține datele de profil, statutul contului (`accessRestricted`), rolul (`Role`) și scorul calculat. Parola este marcată cu `@JsonProperty(access = JsonProperty.Access.WRITE_ONLY)`.
- **TagDTO** — Încapsulează `id`-ul și `label`-ul unui tag.

5.2 Endpoint-uri REST (Controllere)

5.2.1 QuestionController

Gestionează operațiile legate de întrebări. Rutele sunt prefixate cu `/api/questions`.

- **POST** `/api/questions` și `/api/questions/with-images` — Crearea de noi întrebări (cu sau fără atașamente `MultipartFile`).
- **GET** `/api/questions` — Obținerea tuturor întrebărilor. Acceptă un parametru `viewer` pentru a returna starea votului (`like/dislike`) specific acelui utilizator.
- **PUT** `/api/questions/title/{currentTitle}` — Actualizarea unei întrebări, cu verificare de autorizare în serviciu.
- **DELETE** `/api/questions/title/{title}` — Ștergerea unei întrebări. Ștergerea se realizează în cascadă pentru imagini, răspunsuri și voturi asociate.
- **PUT** `/api/questions/{id}/like` și `/dislike` — Acordarea sau anularea unui `like/dislike`. Utilizatorii nu pot vota propriile întrebări.

5.2.2 AnswerController

Logica este similară cu `QuestionController`, rutele fiind prefixate cu `/api/answers`.

- **GET** `/api/answers/question/{questionId}` — Obținerea tuturor răspunsurilor pentru o anumită întrebare.
- **PUT** `/api/answers/{id}/accept` — Marcarea unui răspuns ca soluție. Schimbă statusul întrebării în `RESOLVED`. Doar autorul întrebării poate efectua această acțiune.

5.2.3 UserController

Expune rutele pentru gestiunea profilurilor la `/api/users`.

- **GET** `/api/users/{username}` — Obține profilul unui utilizator.
- **PUT** `/api/users/{username}/block` și `/unblock` — Rute protejate, accesibile doar de utilizatori cu rol de **ADMIN**.
- **GET** `/api/users/{username}/score` — Endpoint pentru preluarea scorului recalcultat dinamic.

5.2.4 AuthController

Gestionează rutele publice la `/api/auth`.

- **POST** `/api/auth/register` — Înregistrează un utilizator nou, hash-uind parola și setând rolul `USER`.

- **GET /api/auth/login** — Returnează datele utilizatorului dacă header-urile de Basic Auth sunt valide.

6 Aplicația Frontend (Angular)

Frontend-ul este realizat folosind **Angular 21** pe o arhitectură curată, bazată exclusiv pe **Standalone Components** (fără a utiliza `NgModules`). Pentru managementul stării și actualizări reactive în UI, se folosesc extensiv **Signals** introduse în versiunile recente de Angular.

6.1 Structura și Rutare

Arhitectura proiectului Angular este organizată pe *feature-uri*:

- **auth**: Logica de login, înregistrare, guards și interceptori.
- **questions**: Lista de întrebări, filtrare, vizualizare generală.
- **answers**: Pagină de detaliu pentru o întrebare, unde utilizatorii pot vedea/adăuga răspunsuri.
- **users**: Vizualizarea profilelor de utilizator și lista de utilizatori pentru administratori.
- **shared/components**: Componente reutilizabile (e.g., `Navbar`).

Garda (*Guard*) `authGuard` protejează rutele interne (redirecționând vizitatorii neautentificați către pagina de login), în timp ce `guestGuard` asigură că utilizatorii logați nu mai pot accesa pagina de register/login, direcționându-i direct la fluxul de întrebări.

6.2 Componente Principale

6.2.1 QuestionList

Reprezintă vizualizarea centrală (dashboard-ul) aplicației.

- **Semnale (Signals)**: Utilizează `signal()` pentru state (`questions`, `isLoading`, texte pentru filtre etc.).
- **Computed Signals**: Filtrarea întrebărilor după titlu, tag, utilizator sau checkbox-ul „Only My Questions” este realizată automat prin `computed()`, garantând actualizări UI performante fără manipulare manuală a datelor.
- Include posibilitatea de a previzualiza imaginile încărcate *inline* sau *fullscreen*.

6.2.2 AnswersList

Reprezintă vizualizarea în detaliu a unei întrebări.

- Răspunsurile sunt afișate sortate după **scorul calculat** (like-uri minus dislike-uri), ordonate descrescător.
- Răspunsul acceptat ca soluție poartă o insignă de validare distinctă, indicând că întrebarea are statusul **RESOLVED**.

6.2.3 AuthInterceptor

Tot traficul de rețea generat de către servicii traversează `authInterceptor`. Acesta injectează **token-ul de Basic Authentication** din `sessionStorage` pe cererile protejate, ignorând rutele de autentificare publice.

7 Servicii de Notificare: SendGrid și Twilio

Blocarea utilizatorilor este însoțită de notificări out-of-band trimise prin servicii Cloud. Pentru a nu afecta timpul de răspuns la login, `NotificationService` are metode marcate cu `@Async`.

- **SendGrid (E-mail):** Utilizează librăria `sendgrid-java`. Când o încercare de login eșuează pe motiv de cont blocat (`accessRestricted == true`), sistemul compune automat un email scurt de avertizare și folosește instanțierea cu `SendGridApiKey` configurată.
- **Twilio (SMS):** Utilizează SDK-ul Twilio (`twilio`). Inițializarea se face pe baza contului SID și `Auth Token`. Funcția creează un mesaj (`Message.creator`) direcționat către numărul de telefon înregistrat în contul utilizatorului blocat.

Orice excepție aruncată în comunicarea cu aceste rețele externe este prinsă și logată, prevenind căderea procesului de validare a credentialelor.

8 Integrare Microservicii (Backend-Second)

Pentru a demonstra principiile arhitecturii bazate pe microservicii, logica de gestiune a etichetelor (**Tags**) nu este efectuată direct de serverul principal.

- **Microserviciul backend-second** rulează distinct pe portul 8081. Are o copie a entității `Tag` și `TagController`.
- `TagMicroserviceClient` (din backend) folosește clasa `RestTemplate` din ecosistemul Spring pentru a efectua apeluri REST sincrone către serviciul **backend-second** (`http://localhost:8081/api/tags`). Toate metodele HTTP (POST, GET, PUT, DELETE) sunt rutate prin intermediul acestui client.
- Această separare permite decuplarea ciclului de viață și eventuala scalare izolată a serviciului care gestionează taxonomia aplicației.

9 Concluzii

Forum Application implementează o soluție software coerentă, cu acoperire mare pe toate fluxurile importante ale unei platforme Q&A: validare robustă, managementul atașamentelor multi-part, autorizări și roluri bine definite, alături de un calcul asincronizat al scorurilor și testare E2E. Tehnologiile alese (Spring Boot 3 și Angular 21 cu Signals) corespund practicilor moderne din industrie pentru realizarea aplicațiilor de nivel Enterprise.